

# Windows 系统 Socket 编程

## ✦ 一、Windows 下 Socket 编程的完整流程

服务器端：

```
WSAStartup() → socket() → bind() → listen() → accept() → send()/recv() → closesocket() → WSACleanup()
```

客户端：

```
WSAStartup() → socket() → connect() → send()/recv() → closesocket() → WSACleanup()
```

## ⚙ 二、WinSock 初始化与清理

在使用任何 socket 函数之前，必须先初始化 WinSock 库。

### 1 WSAStartup()

**作用：**初始化 Windows 套接字库 (Winsock) 。  
(Linux 无需此步骤，Windows 必须要有，否则所有 socket 调用都会失败)

```
int WSAStartup(WORD wVersionRequested, LPWSADATA lpWSADATA);
```

参数：

| 参数                | 含义                                           |
|-------------------|----------------------------------------------|
| wVersionRequested | 指定所需的 Winsock 版本，一般用 MAKEWORD(2, 2) 表示版本 2.2 |
| lpWSADATA         | 指向 WSADATA 结构的指针，用于获取系统支持的 Winsock 版本等信息     |

**WSADATA 结构示例：**

```
typedef struct WSADATA {  
    WORD        wVersion;           // 当前使用的 winsock 版本  
    WORD        wHighVersion;      // 系统支持的最高版本  
    char        szDescription[257];  
    char        szSystemStatus[129];  
    unsigned short iMaxSockets;  
    unsigned short iMaxUdpDg;  
    char FAR     *lpVendorInfo;  
} WSADATA;
```

返回值：

| 返回值 | 含义                                         |
|-----|--------------------------------------------|
| 0   | 成功                                         |
| 非 0 | 失败，可用 <code>WSAGetLastError()</code> 获取错误码 |

示例：

```
WSADATA wsaData;
if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
    printf("WSAStartup failed: %d\n", WSAGetLastError());
    return 1;
}
```

2 `WSACleanup()`

**作用：**清理并释放 Winsock 使用的资源。  
应在所有 socket 操作结束后调用。

```
int WSACleanup(void);
```

返回值：

| 返回值                       | 含义                  |
|---------------------------|---------------------|
| 0                         | 成功                  |
| <code>SOCKET_ERROR</code> | 失败（说明仍有未关闭的 socket） |

 三、核心 Socket 函数（Windows版）

3 `socket()`

**作用：**创建套接字。

```
SOCKET socket(int af, int type, int protocol);
```

参数：

| 参数                    | 含义                                                                      |
|-----------------------|-------------------------------------------------------------------------|
| <code>af</code>       | 地址族（ <code>AF_INET</code> 或 <code>AF_INET6</code> ）                     |
| <code>type</code>     | 套接字类型（ <code>SOCK_STREAM</code> 表示 TCP， <code>SOCK_DGRAM</code> 表示 UDP） |
| <code>protocol</code> | 一般填 0（自动选择协议）                                                           |

返回值：

| 返回值                         | 含义                                        |
|-----------------------------|-------------------------------------------|
| <code>SOCKET</code> （非负）    | 成功，返回新建 socket                            |
| <code>INVALID_SOCKET</code> | 失败，可用 <code>WSAGetLastError()</code> 查询原因 |

示例：

```
SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock == INVALID_SOCKET) {
    printf("socket() failed: %d\n", WSAGetLastError());
    WSACleanup();
    return 1;
}
```

4 `bind()`

作用：绑定本地 IP 地址和端口号。

```
int bind(SOCKET s, const struct sockaddr *addr, int namelen);
```

参数：

| 参数                   | 含义                                                          |
|----------------------|-------------------------------------------------------------|
| <code>s</code>       | 套接字句柄                                                       |
| <code>addr</code>    | 地址结构体指针，一般是 <code>(struct sockaddr*)&amp;sockaddr_in</code> |
| <code>namelen</code> | 地址结构体的长度                                                    |

结构体定义：

```
struct sockaddr_in {
    short sin_family;           // 地址族 AF_INET
    u_short sin_port;           // 端口号（网络字节序）
    struct in_addr sin_addr;    // IP 地址
    char sin_zero[8];           // 保留
};
```

返回值：

| 返回值                       | 含义 |
|---------------------------|----|
| <code>0</code>            | 成功 |
| <code>SOCKET_ERROR</code> | 失败 |

示例：

```
struct sockaddr_in server;
server.sin_family = AF_INET;
server.sin_port = htons(8080);
server.sin_addr.s_addr = INADDR_ANY;

if (bind(sock, (struct sockaddr*)&server, sizeof(server)) == SOCKET_ERROR) {
    printf("bind failed: %d\n", WSAGetLastError());
    closesocket(sock);
    WSACleanup();
    return 1;
}
```

## 5 listen()

作用：让 TCP 套接字进入监听状态。

```
int listen(SOCKET s, int backlog);
```

参数：

| 参数      | 含义      |
|---------|---------|
| s       | 套接字     |
| backlog | 最大等待连接数 |

返回值：

| 返回值          | 含义 |
|--------------|----|
| 0            | 成功 |
| SOCKET_ERROR | 失败 |

## 6 accept()

作用：从监听队列中取出一个待连接请求，建立连接。

```
SOCKET accept(SOCKET s, struct sockaddr *addr, int *addrlen);
```

参数：

| 参数   | 含义        |
|------|-----------|
| s    | 监听 socket |
| addr | 返回客户端地址信息 |

| 参数                   | 含义                   |
|----------------------|----------------------|
| <code>addrLen</code> | 输入输出参数：传入结构大小，返回实际大小 |

返回值：

| 返回值                         | 含义 |
|-----------------------------|----|
| 新的 <code>SOCKET</code>      | 成功 |
| <code>INVALID_SOCKET</code> | 失败 |

示例：

```
struct sockaddr_in client;
int len = sizeof(client);
SOCKET clientSock = accept(sock, (struct sockaddr*)&client, &len);

if (clientSock == INVALID_SOCKET) {
    printf("accept failed: %d\n", WSAGetLastError());
    closesocket(sock);
    WSACleanup();
    return 1;
}
printf("Connected: %s:%d\n", inet_ntoa(client.sin_addr), ntohs(client.sin_port));
```

7 `connect()`

作用：客户端向服务器发起连接请求。

```
int connect(SOCKET s, const struct sockaddr *name, int namelen);
```

参数：

| 参数                   | 含义    |
|----------------------|-------|
| <code>s</code>       | 套接字   |
| <code>name</code>    | 服务器地址 |
| <code>namelen</code> | 地址长度  |

返回值：

| 返回值                       | 含义 |
|---------------------------|----|
| <code>0</code>            | 成功 |
| <code>SOCKET_ERROR</code> | 失败 |

示例：

```
struct sockaddr_in server;
server.sin_family = AF_INET;
server.sin_port = htons(8080);
server.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(sock, (struct sockaddr*)&server, sizeof(server)) == SOCKET_ERROR) {
    printf("connect failed: %d\n", WSAGetLastError());
    closesocket(sock);
    WSACleanup();
    return 1;
}
```

## 8 send() 和 recv()

作用：在已连接的套接字上收发数据。

```
int send(SOCKET s, const char *buf, int len, int flags);
int recv(SOCKET s, char *buf, int len, int flags);
```

参数：

| 参数    | 含义    |
|-------|-------|
| s     | 套接字   |
| buf   | 数据缓冲区 |
| len   | 缓冲区长度 |
| flags | 通常为 0 |

返回值：

| 返回值          | 含义          |
|--------------|-------------|
| >0           | 成功发送/接收的字节数 |
| 0            | 连接关闭        |
| SOCKET_ERROR | 失败          |

示例：

```
char buf[1024];
send(clientSock, "Hello!", 6, 0);

int n = recv(clientSock, buf, sizeof(buf), 0);
if (n > 0) {
    buf[n] = '\0';
    printf("Client says: %s\n", buf);
}
```

## 9 sendto() 和 recvfrom() (UDP)

```
int sendto(SOCKET s, const char *buf, int len, int flags,
           const struct sockaddr *to, int tolen);
int recvfrom(SOCKET s, char *buf, int len, int flags,
             struct sockaddr *from, int *fromlen);
```

示例：

```
sendto(sock, "hello", 5, 0, (struct sockaddr*)&client, sizeof(client));
recvfrom(sock, buf, sizeof(buf), 0, (struct sockaddr*)&client, &len);
```

## 10 closesocket()

作用：关闭 socket（Windows 专用函数）。

```
int closesocket(SOCKET s);
```

| 返回值          | 含义 |
|--------------|----|
| 0            | 成功 |
| SOCKET_ERROR | 失败 |

⚠ 注意：

Windows 不能用 `close()` 来关闭 socket，否则会造成资源泄漏。  
在 Linux 中使用 `close()`，但在 Windows 必须用 `closesocket()`。

## 🧠 四、错误处理函数

### WSAGetLastError()

获取最近一个 Winsock 错误代码（类似于 Linux 的 `errno`）。

```
int err = WSAGetLastError();
printf("Error code: %d\n", err);
```

常见错误码：

| 错误码   | 含义                              |
|-------|---------------------------------|
| 10013 | 权限被拒绝                           |
| 10048 | 地址已被占用 (Address already in use) |
| 10061 | 连接被拒绝 (Connection refused)      |
| 10060 | 连接超时                            |
| 10054 | 连接被远程主机重置                       |

## 五、字节序与地址转换函数（与 Linux 相同）

| 函数                        | 含义                 |
|---------------------------|--------------------|
| htons()                   | 主机序 → 网络序 (short)  |
| htonl()                   | 主机序 → 网络序 (long)   |
| ntohs()                   | 网络序 → 主机序 (short)  |
| ntohl()                   | 网络序 → 主机序 (long)   |
| inet_addr("127.0.0.1")    | IP 字符串 → 32 位网络字节序 |
| inet_ntoa(in_addr)        | 网络字节序 → 字符串        |
| inet_pton() / inet_ntop() | IPv4/IPv6 通用地址转换   |

## 六、完整 TCP 示例（Windows 版）

### ◆ 服务器端：

```
#include <winsock2.h>
#include <stdio.h>
#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsaData;
    WSStartup(MAKEWORD(2,2), &wsaData);

    SOCKET serverSock = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in server;
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = INADDR_ANY;

    bind(serverSock, (struct sockaddr*)&server, sizeof(server));
    listen(serverSock, 5);
}
```



```

struct sockaddr_in client;
int len = sizeof(client);
SOCKET clientSock = accept(serverSock, (struct sockaddr*)&client, &len);

char buf[512];
recv(clientSock, buf, sizeof(buf), 0);
printf("Client says: %s\n", buf);

send(clientSock, "Hello from server!", 18, 0);

closesocket(clientSock);
closesocket(serverSock);
WSACleanup();
return 0;
}

```

## ◆ 客户端:

```

#include <winsock2.h>
#include <stdio.h>
#pragma comment(lib, "ws2_32.lib")

int main() {
    WSADATA wsaData;
    WSStartup(MAKEWORD(2,2), &wsaData);

    SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in server;
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(sock, (struct sockaddr*)&server, sizeof(server));

    send(sock, "Hi Server!", 10, 0);

    char buf[512];
    int n = recv(sock, buf, sizeof(buf), 0);
    buf[n] = '\0';
    printf("Server says: %s\n", buf);

    closesocket(sock);
    WSACleanup();
    return 0;
}

```

## 七、总结：Windows 与 Linux Socket 编程主要区别

| 项目    | Windows                         | Linux                                                                |
|-------|---------------------------------|----------------------------------------------------------------------|
| 初始化   | 需要 <code>WSAStartup()</code>    | 不需要                                                                  |
| 清理    | <code>WSACleanup()</code>       | 不需要                                                                  |
| 关闭函数  | <code>closesocket()</code>      | <code>close()</code>                                                 |
| 错误获取  | <code>WSAGetLastError()</code>  | <code>errno</code>                                                   |
| 套接字类型 | <code>SOCKET</code> (无符号)       | <code>int</code>                                                     |
| 库依赖   | <code>ws2_32.lib</code>         | 无需手动链接                                                               |
| 头文件   | <code>&lt;winsock2.h&gt;</code> | <code>&lt;sys/socket.h&gt;</code> 、 <code>&lt;arpa/inet.h&gt;</code> |